DOCINFERX : LOCAL AI-POWERED DOCUMENT INTELLIGENCE SYSTEM

STUDENT: KRISHNA ARORA

REG. NO.: 25BAC10022

COURSE: FUNDAMENTALS IN AI AND ML

FACULTY: DR. VINESH KUMAR

INSTITUTE: VIT BHOPAL UNIVERSITY

SEMESTER: 2ND SEM

SUBMISSION DATE: 29.3.2026

INTRODUCTION

- Modern documents contain a large amount of unstructured information spread across PDFs, images, scanned pages. Extracting meaning from these documents manually is slow and inefficient.

- DocInferX is a local RAG-based system that allows users to upload documents, process them using OCR and embeddings, and then ask natural-language questions to retrieve precise answers entirely offline on local machine.

- The project demonstrates real-world application of NLP, vector databases, and information retrieval.

PROBLEM STATEMENT

Individuals and professionals struggle to extract useful information from large multi-page PDFs, scanned documents, and images. Searching manually is slow, error-prone, and computationally expensive. There is a need for a local, private, offline system that can:

- Ingest PDFs and images
- Extract and clean text
- Convert content into embeddings for fast retrieval
- Answer user questions accurately
- Provide a modern, intuitive interface

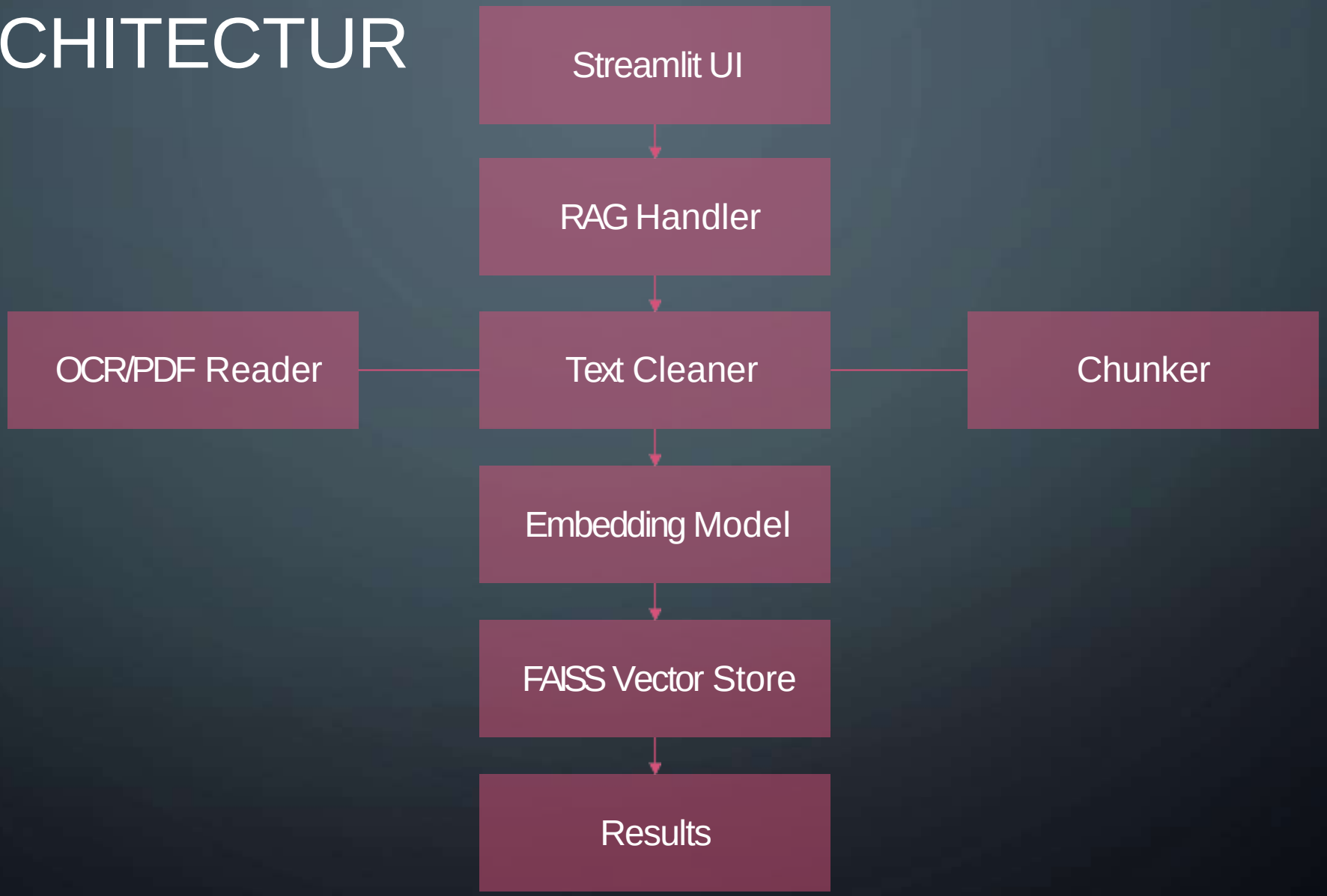
FUNCTIONAL REQUIREMENTS

1. Document Ingestion Module
   - Accepts PDF or image files
   - Extracts text via PDF parser or OCR
   - Cleans and preprocesses content
2. Embedding & Indexing Module
   - Splits text into chunks
   - Generates vector embeddings using Phi-2
   - Stores them in FAISS vector store
3. RAG Query Module
   - Accepts natural-language queries
   - Retrieves relevant chunks
   - Generates response using LLM
   - Displays clean, readable answers

NON-FUNCTIONAL REQUIREMENTS

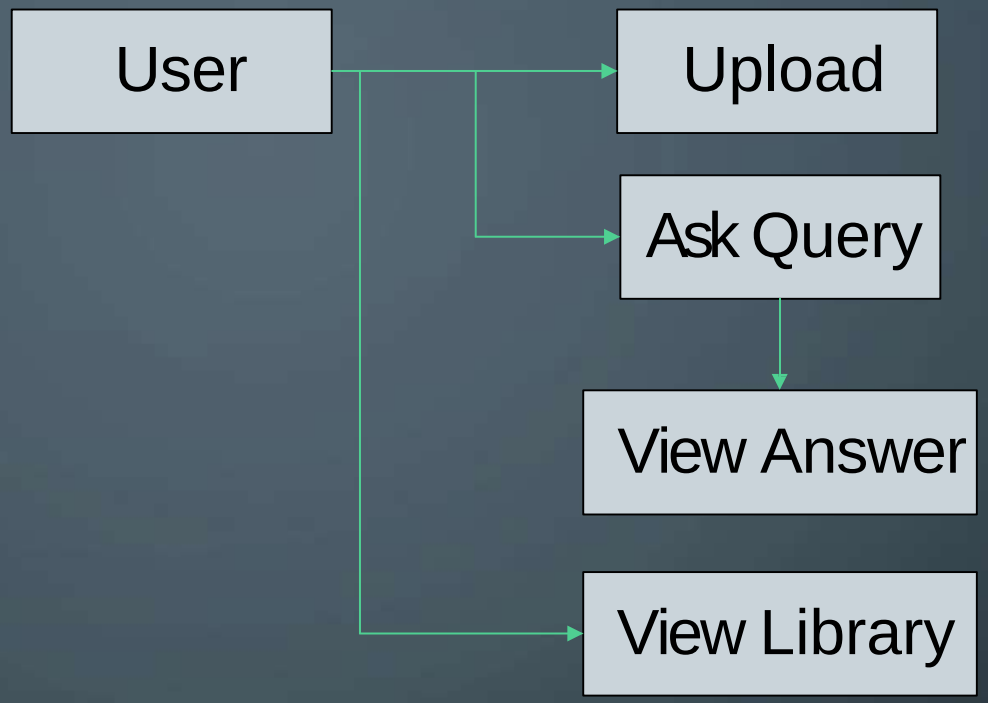
1. **Usability** – Clean Streamlit UI with modern design.
2. **Performance** – FAISS vector search ensures fast retrieval even for large documents.
3. **Maintainability** – Modular folder structure with separate components.
4. **Reliability** – Handles corrupted PDFs, missing pages, and OCR failures.
5. **Resource Efficiency** – Runs fully offline with GPU requirement.
6. **Security** – No cloud upload; all processing is local.

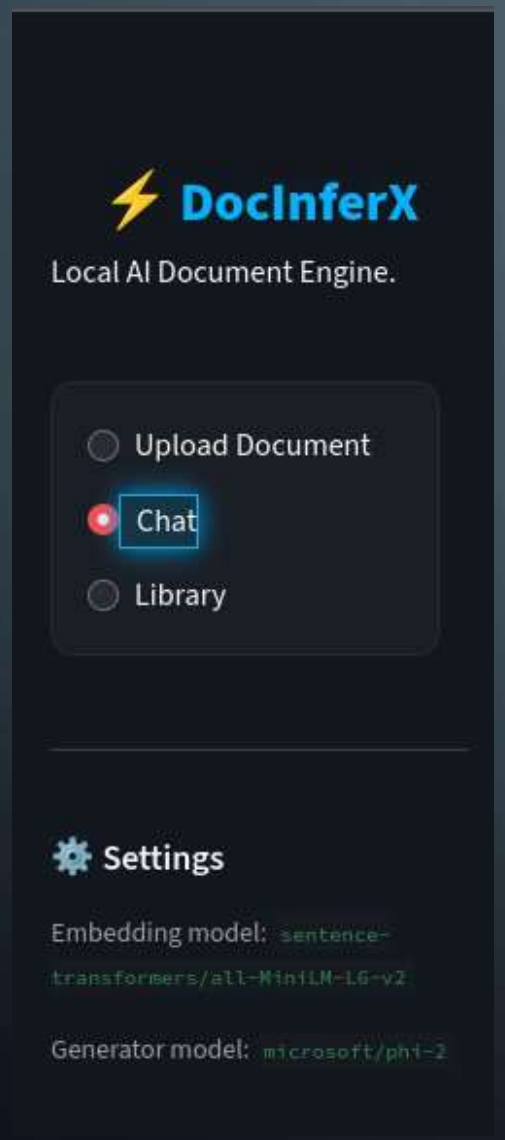
SYSTEM ARCHITECTURE:



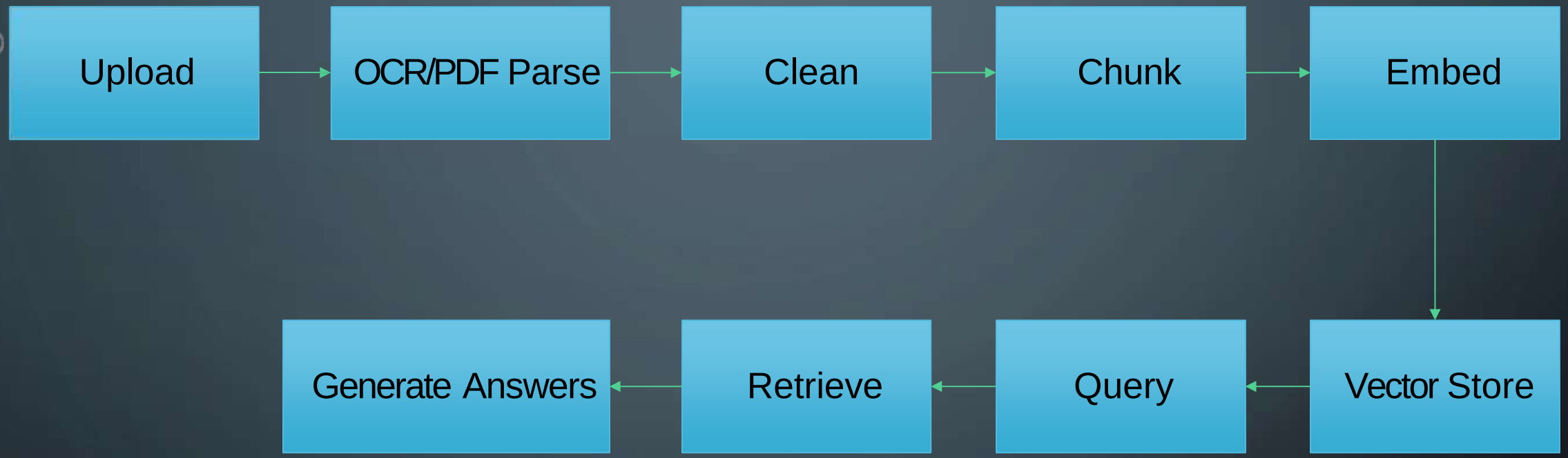
Design Diagrams

Use Case Diagram



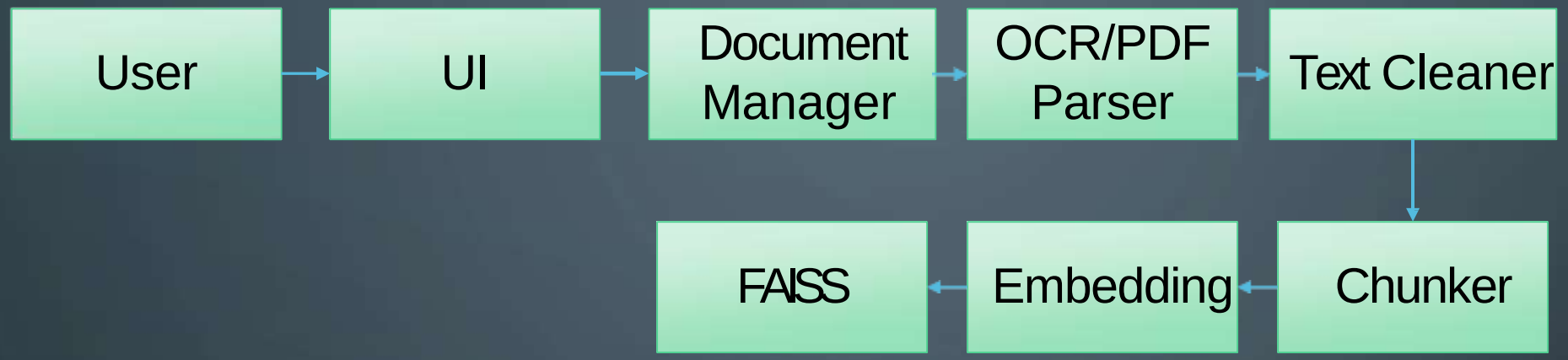


- Workflow Diagram

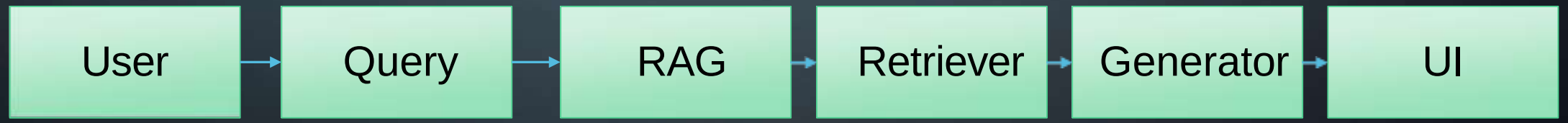


- Sequence Diagram

1.



2.



- Class/Component Diagrams

```
DocInferX/
│
├─── streamlit_app.py      # Main UI
├─── config.py             # Global config
├─── cli_test.py           # CLI test script
│
├─── embed/
│    ├─── vectorizer.py     # Embeddings
│    └─── chunker.py        # Text chunking
│
├─── preprocess/
│    ├─── pdf_reader.py     # PDF parsing
│    ├─── ocr.py            # OCR engine
│    └─── cleaner.py        # Text cleanup
│
├─── library/
│    ├─── manager.py        # Document ingestion
│    └─── metadata.py       # Meta store
│
├─── rag/
│    └─── pipeline.py       # RAG inference
│
├─── Samples/              # Example files
│
├─── data/                 # Vector store + metadata
│
├─── requirements.txt
└─── Dockerfile
```

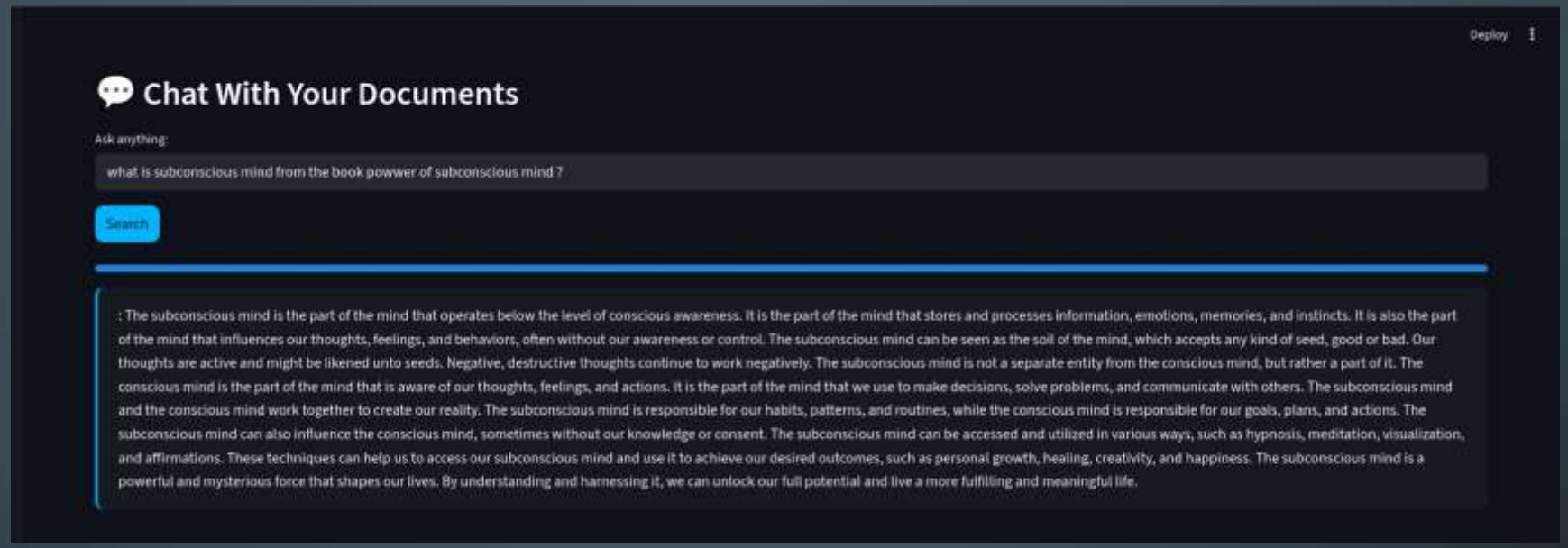
DESIGN DECISIONS & RATIONALE

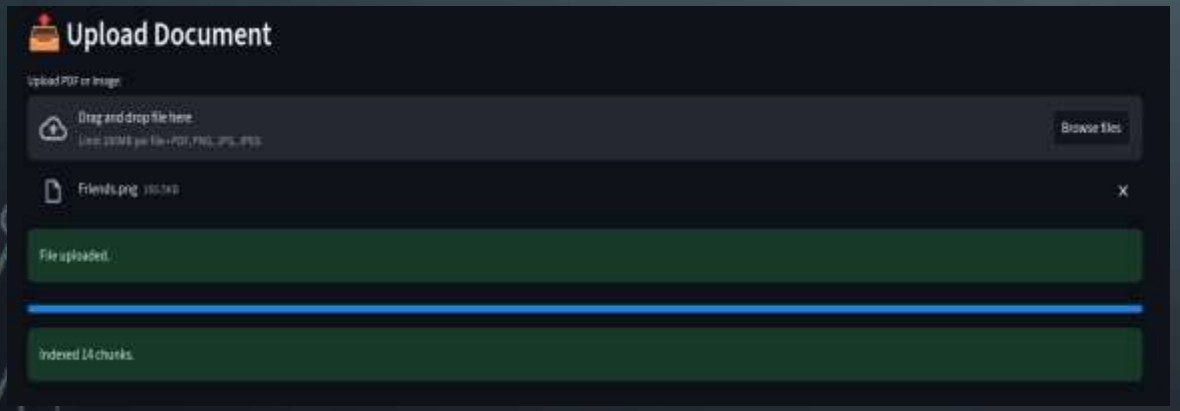
1. **Local-first architecture:** Ensures privacy and offline access.
2. **FAISS:** Fast similarity search for large text corpora.
3. **Phi-2:** Lightweight and performant LLM suitable for on-device generation.
4. **Streamlit:** Simplifies UI without needing frontend frameworks.
5. **Modular folder structure:** Easier debugging and future scalability.

Implementation Details

- **Python backend with separate packages**:
  - /preprocess (OCR, cleaning, PDF reading)
  - /embed (chunker, vectorizer)
  - /rag (retrieval pipeline)
  - /library (metadata and document management)
- **UI built in Streamlit**
- **Vector DB: FAISS**
- **OCR: PaddleOCR**
- **Embeddings: Phi-2**
- **Storage: Local JSON metadata + FAISS index**

Screenshots





Testing Approach

- Tested with multiple PDFs: books, handwritten scans, power-law dense PDFs.
- Tested OCR accuracy on blurred images.
- Verified embedding and chunk alignment.
- Checked retrieval accuracy with different queries.
- Checked UI on Linux + Windows.
- Docker test for portability.

Challenges Faced

- Handling large PDFs with mixed text + images
- Chunking strategy for semantic coherence
- Optimizing OCR speed
- Avoiding cache corruption
- Designing a modern UI in Streamlit
- Mismatch of python modules versions
- VRAM management to make it Local on device

Learnings & Key Takeaways

- Practical understanding of RAG systems
- Using FAISS and embeddings
- Implementing OCR pipelines
- Multi-module project structuring
- Version control workflows (GitHub)
- Dockizing Python applications
- UI/UX considerations even in backend-heavy projects

FUTURE ENHANCEMENTS

- Support for audio transcription
- Summarization mode
- Cloud-sync optional mode
- Fine-tuned local model for document answering
- Increasing response speed
- Reducing hallucinations

References

- Faiss documentation
- HuggingFace Transformers
- PaddleOCR documentation
- Streamlit documentation
- Various research papers on RAG